

1. Informed Search

Answer: C) Greedy Best-First Search

Explanation: Greedy Best-First Search uses only the heuristic value **$h(n)$** to choose the node that looks closest to the goal. It does not consider the cost already travelled.

2. A* Search Evaluation Function

Answer: C) $f(n) = g(n) + h(n)$

Explanation:

- **$g(n)$** = cost from the start to the current node
 - **$h(n)$** = estimated cost from current node to goal
So, **$f(n) = g(n) + h(n)$** .
-

3. Property of Heuristic for A*

Answer: B) Admissibility

Explanation: An admissible heuristic **never overestimates** the actual cost to reach the goal. This helps A* find an optimal solution.

4. Local Search Algorithm

Answer: B) Hill Climbing

Explanation: Hill Climbing is a local search method. It continuously moves from the current state to a better neighboring state.

5. Adversarial Search

Answer: B) Minimax Algorithm

Explanation: Minimax is commonly used in games such as chess and tic-tac-toe. It considers both the **player's move and the opponent's response**.

6. Robot Using Only Estimated Distance

Answer: B) Greedy Best-First Search

Explanation: Greedy Best-First Search selects a node based only on **$h(n)$** , the estimated distance to the goal. Therefore, it can be fast but may produce a non-optimal path.

7. GPS Route Calculation

*Answer: B) A Search**

Explanation: A* combines the distance already travelled **$g(n)$** with the estimated remaining distance **$h(n)$** .

Formula:

$$f(n) = g(n) + h(n)$$

8. University Exam Scheduling

Answer: B) Constraint Satisfaction Problem

Explanation: Exam scheduling has **variables, possible values, and constraints**. For example, two exams should not overlap and room capacity must be satisfied.

9. Chess AI

Answer: C) Minimax Algorithm

Explanation: Minimax evaluates possible moves and considers the opponent's possible responses to select the best move.

10. Robot in Unknown Environment

Answer: B) Online Search Agent

Explanation: An online search agent does not know the complete environment beforehand. It **explores, learns, and makes decisions step-by-step**.

11. Importance of Admissible Heuristic

Answer: B) It ensures optimality

Explanation: An admissible heuristic does not overestimate the actual cost. Therefore, A* can guarantee an **optimal solution** under the appropriate conditions.

12. Benefit of Alpha-Beta Pruning

Answer: B) Reduces number of nodes evaluated

Explanation: Alpha-Beta pruning removes branches that cannot affect the final Minimax decision. This makes the search **faster**.

13. Main Drawback of Hill Climbing

Answer: C) Getting stuck in local maxima

Explanation: Hill Climbing may reach a **local maximum** where no neighboring state is better, even though a better global solution exists.

14. Valid CSP Solution

Answer: C) Constraint satisfaction

Explanation: A CSP solution is valid only when **all variables are assigned values without violating any constraints**.

15. Role of Evaluation Function

Answer: B) Estimate utility of a state

Explanation: In games, an evaluation function estimates how **good or bad a game state** is when the entire game tree cannot be searched.

16. Calculate A* f(n)

Given:

- $g(n) = 7$
- $h(n) = 5$

Formula:

$$f(n) = g(n) + h(n)$$

$$f(n) = 7 + 5 = 12$$

Answer: A) 12

Explanation: A* adds the actual path cost and estimated remaining cost.

17. Heuristic Overestimates Cost

Answer: B) A becomes non-optimal*

Explanation: If the heuristic overestimates the actual cost, it is **not admissible**. A* may still find a solution, but it is no longer guaranteed to find the best solution.

18. Binary Tree Nodes

Given:

- Branching factor **b = 2**
- Depth **d = 4**

Total nodes:

$$1 + 2 + 4 + 8 + 16 = 31$$

Answer: C) 31

Explanation: A full binary tree with depth 4 has **31 total nodes** including the root.

19. Minimax Leaf Nodes

Given:

- Branching factor **b = 3**
- Depth **d = 3**

Formula:

$$\text{Leaf nodes} = b^d$$

$$= 3^3 = 27$$

Answer: B) 27

Explanation: Each level has 3 times more nodes than the previous level, giving **27 leaf nodes** at depth 3.

20. Alpha-Beta Best-Case Complexity

Answer: B) $O(b^{(d/2)})$

Explanation: In the best case, Alpha-Beta pruning can eliminate many unnecessary branches. It reduces the effective search depth roughly by half.

Best-case: $O(b^{(d/2)})$

